CS 409: MP 3

https://gemini.google.com/app/a27c8e27794f6a1a

User prompt: I am going to work on the GET HTTP as I worked on some of it but I stopped since you suggested that DELETE and POST are easier.

Response: Tackling `GET` now makes total sense since `POST` and `DELETE` give you the data to actually test your reads.

For `GET`, keep in mind the two distinct endpoints:

1. `GET /:id` (Single Lookup)

Only handles fetching one document by its ID and supports `select`.

- **Step 1:** Extract `id` from `req.params`.
- **Step 2:** Parse `req.query.select` (if provided). If parsing fails, return `400`.
- **Step 3:** Query with `findById(id).select(...)`.
- **Step 4:**
    - If result is `null` → return `404`.
    - If found → return `200` with the document.
- **Step 5:** In `catch`, if `error.name === 'CastError'` → return `404` (invalid ID format means it does not exist); otherwise return `500`.

2. `GET /` (Collection Query)

Builds a query pipeline step-by-step and supports `where`, `sort`, `select`, `skip`, `limit`, and `count`.

- **Step 1 (Safely parse JSON parameters):** Wrap all `JSON.parse()` calls in a `try/catch` at the very beginning. If any of `where`, `sort`, or `select` contains invalid JSON syntax, immediately return `400 Bad Request`.
- **Step 2 (Initialize the query):** Start the chain with the parsed `where` filter: `let query = Task.find(whereFilter);`
- **Step 3 (Chain conditional options):**
    - If `sort` exists → `query = query.sort(sortOptions)`
    - If `select` exists → `query = query.select(selectOptions)`
    - If `skip` exists → `query = query.skip(parseInt(req.query.skip, 10))`
    - If `limit` exists → `query = query.limit(parseInt(req.query.limit, 10))`
- **Step 4 (Handle `count` flag):**
    - If `req.query.count === 'true'` (or `true`) → use `Task.countDocuments(whereFilter)` instead of executing the full query array, and return `200` with `{ data: count }`.
    - Otherwise → `const results = await query.exec();` and return `200` with `{ data:`

`results }`.

- *Remember:* An empty array `[]` or count of `0` is still a successful `200 OK`.

Immediate Next Steps

1. Implement the JSON parse block with its `400` guard.
2. Build the query chaining sequence on `Task.find()`.
3. Add the `count` condition check right before executing.

User prompt: The parameters that I need to tackle in the GET HTTP are where, sort, select, skip, count, and limit. So in simple terms what needs to be done for the get and how should I get started on this. I might focus on doing the sort parameter under Tasks. Please note that I already worked on some of this for a bit before moving on to something else.

Response: How Collection `GET` Works in Simple Terms

A collection `GET` is a pipeline with three stages:

1. **Parse:** Read incoming strings from `req.query` and convert JSON strings into JavaScript objects. If any JSON string is invalid syntax, return `400 Bad Request`.
2. **Build Query:** Start a Mongoose query (`Task.find()`), then conditionally chain helper methods (`.sort()`, `.select()`, `.skip()`, `.limit()`) onto it.
3. **Execute:** Check whether `count` was requested. If so, return the number; if not, `await` the query and return the array.

Focusing on `sort`: What It Is and How It Works

- **What the client sends:** A JSON string in the query string indicating field names and directions (1 for ascending, -1 for descending).
  - Example: `GET /api/tasks?sort={"deadline": 1}` (earliest deadline first)
  - Example: `GET /api/tasks?sort={"dateCreated": -1}` (newest first)
- **What Mongoose expects:** `.sort()` accepts an object directly, e.g. `.sort({ deadline: 1 })`.

Concrete Implementation Steps for `sort`

1. **Safely extract and parse `req.query.sort`:** Wrap the parsing in a `try...catch`. If `req.query.sort` is provided, parse it with `JSON.parse()`. If parsing throws a `SyntaxError`, send `400`.
2. **Initialize the base query:** Start with `let query = Task.find(whereFilter);` (or `Task.find()` if `where` is not yet supplied).
3. **Chain `.sort()`:** If a valid sort object was parsed, chain it:

```
if (sortOptions) {
  query = query.sort(sortOptions);
}
```

4. **Execute and return:** `const tasks = await query.exec();` and return `200` with `{ data: tasks }`.

Minimal Scaffold

```
router.get('/', async (req, res) => {
  try {
    let sortOptions = {};

    // 1. Parse JSON safely
    if (req.query.sort) {
      try {
        sortOptions = JSON.parse(req.query.sort);
      } catch (err) {
        return res.status(400).json({ message: "Invalid JSON in sort parameter" });
      }
    }

    // 2. Build query
    let query = Task.find();

    if (Object.keys(sortOptions).length > 0) {
      query = query.sort(sortOptions);
    }

    // 3. Execute
    const tasks = await query.exec();
    return res.status(200).json({ data: tasks });
  } catch (err) {
    return res.status(500).json({ message: "Server error", error: err.message });
  }
});
```

Verification Checklist

- [ ] Send a request without `?sort` → returns `200` with all tasks in default order.
- [ ] Send `?sort={"deadline": 1}` → returns `200` ordered from earliest to latest date.
- [ ] Send `?sort={"deadline": -1}` → returns `200` ordered from latest to earliest date.
- [ ] Send malformed JSON like `?sort={deadline:1}` (missing quotes) → returns `400 Bad Request`.

User prompt: You have to do similar check with other parameters to validate that whatever has been passed in is correct and it is good to be used. If there are any errors with the passed values, then you would have to return different errors, 400, 404, etc based on the type of error right? Please try to not give me too much or any code. Thank you.

Response: Yes, that is exactly right. Every query parameter passed by the client must be validated before you apply it to the database query, but keep in mind that **`404` is almost never used for collection query parameters**.

Here is how the errors and validation break down across the parameters:

Why It Is Almost Always `400 Bad Request`

In a collection `GET` (`GET /tasks` or `GET /users`), errors with the passed parameters mean the client sent an invalid, malformed, or unparseable instruction. That is a client-side syntax or format violation, which is strictly

`400 Bad Request`:

1. **`where`, `sort`, `select` (JSON Objects):**
    - They must be valid JSON strings representing objects.
    - If parsing fails (syntax error) or the parsed result is not an object (e.g., passing a plain number or string when an object is expected), fail immediately with `400`.
2. **`skip`, `limit` (Integers):**
    - Must be valid numbers and non-negative integers.
    - If a client passes non-numeric values (like `?limit=abc` or `?skip=-5`), reject with `400`.
3. **`count` (Boolean flag):**
    - Usually checked as a string flag (like `"true"`). If your spec expects strict booleans or JSON, an unparseable or unrecognized value returns `400`.

What About `404 Not Found`?

- **`404` does not apply to zero search results:** If a valid `where` filter finds no matching tasks in the database, that is **not** a `404`. The query succeeded; there are just 0 matching items. The response must be `200 OK` with an empty list `[]` (or count `0`).
- **Where `404` belongs:** `404` is strictly reserved for when a client targets a specific resource URL path that does not exist—specifically `GET /tasks/:id` when that `:id` is not in the database.

What About `500 Internal Server Error`?

- `500` is reserved for unexpected runtime crashes, broken server logic, or losing connection to the database. It is caught by your outer `catch` block when an internal operation fails unexpectedly, not because the user sent bad input.

Validation Checklist Before Applying to Query

1. **Parse & Type-Check:** Verify `where`, `sort`, and `select` parse cleanly into objects, and `skip`/`limit` parse into positive integers. If any check fails → `400`.
2. **Apply to Mongoose:** Chain the valid options onto `Task.find()`.
3. **Execute & Respond:** Return `200` with the results (even if empty).
4. **Outer Catch:** Catch unexpected database/network failures → `500`.

User prompt: So for count right, we never care about some of these parameters: sort, select, and limit. All we care about is where right? Or do you just want us to have all those parameters and then count it?

Response: You only care about **`where`**.

When `count=true` is requested, you completely ignore `sort`, `select`, `skip`, and `limit`.

Why the Others Don't Apply

- **`sort`:** Ordering the documents does not change how many there are.

- **`select`:** Choosing which fields to return is irrelevant because you are returning a number, not documents.
- **`limit` and `skip`:** `count` represents the total number of documents matching the `where` filter, not the size of a paginated page. If you have 50 pending tasks and a client asks for `?count=true&limit=10`, the answer is still 50.

How to Execute It

Instead of running a full query with `.find()`, check if `count` is enabled and route straight to Mongoose's dedicated counting method:

1. Parse your `where` parameter into a filter object (defaulting to `{}` if none is provided).
2. If `req.query.count === 'true'` (or `true`):
    - Run `await Task.countDocuments(whereFilter)`.
    - Return `200` with that count.
3. If `count` is not requested:
    - Build the standard chain (`.find(whereFilter)`, `.sort()`, `.select()`, `.skip()`, `.limit()`).
    - Await the results and return the list.

Verification Steps

- [ ] Query with `?count=true` and no filters → returns the total number of tasks in the database.
- [ ] Query with `?count=true&where={"completed":true}` → returns only the count of completed tasks.
- [ ] Query with `?count=true&limit=5` → still returns the full count matching the filter, completely ignoring the `limit`.

User prompt: Hmmm. I see. I got a bit confused on this. Thank you.

Response: It is a very common point of confusion, especially since query strings can pack so many parameters together.

To keep your implementation clean, here is the concrete execution order:

1. **Parse & Validate All Parameters First:**
    - Extract `where`, `sort`, `select`, `skip`, `limit`, and `count`.
    - Safely parse JSON strings inside a `try/catch`. If any JSON is malformed or invalid → return **`400 Bad Request`**.
2. **Branch on `count`:**
    - If `count=true`, short-circuit directly to `Task.countDocuments(whereFilter)` (or `User.countDocuments(whereFilter)`), await the integer, and return **`200 OK`**.
3. **Otherwise, Build the Standard Document Pipeline:**
    - Start with `Model.find(whereFilter)`.

- Conditionally chain `.sort()`, `.select()`, `.skip()`, and `.limit()`.
- Execute the query with `await` and return `200 OK` with the resulting array.

User prompt: I think I am successfully finished getting GET, POST, and DELETE HTTPs done. I am only left with PUT right now. Is there any suggestions that you would like to suggest for this MP?

Response: Finishing `GET`, `POST`, and `DELETE` knocks out the bulk of the query building and schema plumbing. `PUT /:id` is the final core operation, and there are a few subtle catches specific to Mongoose and relational tracking you will want to handle.

Key Considerations for `PUT /:id`

1. Full Replacement vs. Partial Update

- Strict REST semantics treat `PUT` as a complete replacement of the resource, whereas `PATCH` is for partial updates.
- Check your assignment spec: if all fields (or all required fields) must be present in `req.body`, validate that first. If any required field is missing, return `400 Bad Request`.

2. Essential Mongoose Flags

If using `findByIdAndUpdate(id, req.body, options)`, you must supply two critical options:

- `{ new: true }`: By default, Mongoose returns the document *before* the update was applied. Setting `new: true` ensures the returned document is the newly updated one.
- `{ runValidators: true }`: By default, Mongoose update operations **skip** schema validators. You must enable this flag so missing required fields, enum violations, or type mismatches trigger a `ValidationError` (which you can catch and map to `400`).

3. Task Assignment State Synchronization

If updating a **Task**, changes to either `completed` or `assignedUser` require keeping the related user's `pendingTasks` array in sync:

- **User Reassignment:** If a task changes from User A to User B, remove the task's ID from User A's `pendingTasks` and add it to User B's `pendingTasks`. Verify that User B actually exists before committing the update (return `404` or `400` per spec if not).
- **Completion Status Change:** If a task is marked `completed: true`, remove its ID from the assigned user's `pendingTasks`. If it is toggled back to `completed: false`, add it back to `pendingTasks`.

4. Handling `404` vs. `400`

- Check if the document exists: if `findByIdAndUpdate` returns `null`, return `404 Not Found`.
- In your `catch` block, map `CastError` (an invalid ObjectId string format) to `404 Not Found` (or `400` depending on your test runner's expectations), and map `ValidationError` to `400 Bad Request`.

Step-by-Step Verification Checklist

1. **Verify Baseline Replacement:**

- Send a valid update payload to an existing ID → returns `200 OK` with the updated document reflecting the new values.

2. **Verify Non-Existent ID:**
   - Send a valid payload to a random 24-character hex ID that does not exist → returns `404 Not Found`.
3. **Verify Validation Enforcement:**
   - Send an invalid payload (e.g., missing a required field or providing a malformed date/type) → returns `400 Bad Request`.
4. **Verify Relational Consistency (Tasks):**
   - Mark a task as completed → check the assigned user via `GET /users/:id` to confirm the task ID was removed from `pendingTasks`.
   - Reassign a task to another user → verify the old user lost the ID and the new user gained it.

User prompt: Okay. I was able to understand something from here. So based on this, what I can say is this(note, I am only going to focus on the tasks/:id for now): Always best to validate the given id to see if a task exists or not. If not, then we need to return a status error code. Then we will need to validate the new data in there and see if it is valid or not before switching it in with the existing data right?

Response: Yes, that is the exact conceptual sequence.

Breaking it down step-by-step:

1. ID Verification & Existence Check

Before modifying anything, verify the target task exists:

- Extract the ID from the URL.
- If the task does not exist in the database, stop immediately and return `404 Not Found`.
- This also gives you access to the **current state** of the task (e.g., who was previously assigned to it, whether it was already marked completed), which is crucial for side-effect cleanups.

2. Payload Validation

Before applying changes to the database:

- Check that the required fields (e.g., `name`, `deadline`) are present in `req.body`. If anything required is missing or empty, abort and return `400 Bad Request`.
- If an `assignedUser` ID is provided in the new payload, check whether that user actually exists in the database. If not, abort with `404 Not Found` (or `400`, depending on your assignment's specific rule for non-existent assigned users).

3. Replace & Sync Relational State

Once both the target task and the incoming data are verified:

- Apply the updates to the task document.

- Compare the old task state with the new task state to handle the user's `pendingTasks` array:
  - **Was it reassigned?** Remove the task ID from the previous user's `pendingTasks`, and add it to the new user's `pendingTasks`.
  - **Was completion status changed?** If changed to `completed: true`, remove the task ID from the assigned user's `pendingTasks`. If reverted to `false`, add it back.
- Save the updated task and return `200 OK` with the updated task object.

Immediate Execution Order

1. Fetch existing task by ID → if null, return `404`.
2. Validate required body fields → if invalid, return `400`.
3. Validate new `assignedUser` exists (if present) → if not found, return `404`.
4. Apply updates and synchronize `pendingTasks` on the user model.
5. Return `200` with the updated task.

User prompt: So let's say that you get the id, pull out the task and store it in a variable called a. You use req.body to obtain all the inputs from the user. Once that is validated, you just replace the values in the existing task or do you have to delete it and then create an entire new task?

Response: You **replace the values in the existing task**; you do not delete and recreate it.

Deleting and recreating it generates a brand-new MongoDB `_id` and resets the original `dateCreated` timestamp, which breaks existing references and fails REST guarantees for updating a specific resource.

How to Update Using Your Variable `a`

Because `a` is a Mongoose document fetched via `findById`, you can overwrite its properties directly or use `Object.assign()`, then call `.save()`:

1. **Overwrite the fields on `a`:**

   ```
   a.name = req.body.name;
   a.description = req.body.description;
   a.deadline = req.body.deadline;
   a.completed = req.body.completed;
   a.assignedUser = req.body.assignedUser;
   ```

   (Or concisely: `Object.assign(a, req.body)`)

2. **Run side-effect checks:** Compare the previous values (like the old `assignedUser` or old `completed` status) with the new values to update the respective user's `pendingTasks` array.

3. **Save the document:**

   ```
   const updatedTask = await a.save();
   return res.status(200).json({ message: "Task updated", data: updatedTask });
   ```

Why Updating `a` and Calling `.save()` Is Better

- **Preserves Identity:** The task keeps its original `_id` and original creation timestamp.
- **Preserves Old Values for Comparison:** Having `a` in memory before mutating it lets you inspect `a.assignedUser` and `a.completed` to determine whether you need to pull or push task IDs from users' `pendingTasks`.
- **Automatic Validation:** Calling `await a.save()` automatically runs all schema validations on the updated fields, throwing a `ValidationError` if any required inputs are invalid so your `catch` block can return `400`.

Immediate Action Checklist

- [ ] Read existing task into `a = await Task.findById(req.params.id)`. If `!a`, return `404`.
- [ ] Verify payload inputs (`name`, `deadline`). If invalid, return `400`.
- [ ] Note the previous values: `const oldUser = a.assignedUser; const oldCompleted = a.completed;`.
- [ ] Mutate `a` with the new body data.
- [ ] Synchronize `pendingTasks` for `oldUser` and `req.body.assignedUser` based on the status change.
- [ ] Call `await a.save()` and return `200`.

User prompt: So when working on this assignment, you in general do not want any duplicate emails right? Do you have to catch the ones that might be already caught by something like MongoDB or Render since I am working on this assignment using those?

Response: Yes, you definitely do not want duplicate emails. However, **Render does nothing here** (it is just running your Node server), and **MongoDB handles the actual rejection, but you must catch it in your Express code so it returns `400` instead of crashing or sending `500`**.

Here is how the chain works and what you need to handle:

1. Where the Enforcement Happens (MongoDB)

In your `UserSchema`, setting `unique: true` on the `email` field creates a **unique index** inside MongoDB:

```
email: {
  type: String,
  required: true,
  unique: true
}
```

When someone attempts to save a user with an email that already exists, MongoDB aborts the write and throws a specific duplicate-key error.

2. The Catch: MongoDB's Error Code `11000`

A duplicate email is **not** a standard Mongoose `ValidationError`. Instead, MongoDB throws an error with the code `11000`:

- Standard validation errors (missing name, missing email) show up as `error.name === 'ValidationError'`.

- Duplicate unique index violations show up as `error.code === 11000`.

If you only check for `error.name === 'ValidationError'`, a duplicate email error will slip past your check, hit your fallback handler, and return **`500 Internal Server Error`**—which causes assignment test suites to fail.

3. How to Route It Cleanly

You have two simple ways to handle it:

Option A: In the `catch` block (Standard approach)

Listen for `error.code === 11000` alongside `ValidationError` and map it to **`400 Bad Request`**:

```
try {
  // your save or update logic
} catch (error) {
  if (error.name === 'ValidationError' || error.code === 11000) {
    return res.status(400).json({
      message: "Validation Error: Duplicate email or missing required fields",
      error: error.message
    });
  }
  return res.status(500).json({ message: "Server error", error: error.message });
}
```

Option B: Pre-check before saving (Defensive approach)

Before calling `.save()`, query the database to see if that email already exists:

```
const existingUser = await User.findOne({ email: req.body.email });
if (existingUser) {
  return res.status(400).json({ message: "Email already exists" });
}
```

*(If doing this on `PUT /users/:id`, make sure you check if an existing user was found **with a different `_id`**, so a user updating their name without changing their email doesn't trigger a false alarm).*

Checklist

- [ ] Ensure `unique: true` is on `email` in your `UserSchema`.
- [ ] Make sure your `POST /users` and `PUT /users/:id` routes return **`400 Bad Request`** when an email collision occurs (either via pre-check or checking `error.code === 11000`).

User prompt: What is with option A here: Option A: In the catch block (Standard approach) Listen for error.code === 11000 alongside ValidationError and map it to 400 Bad Request: JavaScript try { // your save or update logic } catch (error) { if (error.name === 'ValidationError' ll error.code === 11000) { return res.status(400).json({ message: "Validation Error: Duplicate email or missing required fields", error: error.message }); } return res.status(500).json({ message: "Server error", error: error.message }); }

Response: Why Option A Is Structured This Way

When you interact with MongoDB through Mongoose, errors originate from two distinct layers: **Mongoose itself** and the **underlying MongoDB engine**.

1. The Two Different Errors

Layer 1: Mongoose Schema Validation (`error.name === 'ValidationError'`)

- **Where it runs:** Locally inside your Node.js application process, before any network packet reaches MongoDB.
- **What triggers it:** Failing client-side schema constraints like `required: true`, `minlength`, or passing a string into a date/number field.
- **How Mongoose flags it:** Mongoose formats this error object with the property `error.name = 'ValidationError'`.

Layer 2: MongoDB Database Engine (`error.code === 11000`)

- **Where it runs:** Inside the actual MongoDB database instance on disk.
- **What triggers it:** The `unique: true` property. `unique` is **not** a Mongoose validator—it is an instruction telling MongoDB to build a unique B-Tree index on that collection.
- **The conflict:** Mongoose does not know if an email is a duplicate until it sends the write command across the network. When MongoDB attempts to insert the document and finds the index key already occupied, the database rejects the write and sends back an engine-level error code: **`11000`** (`E11000 duplicate key error collection`).
- **How Mongoose flags it:** Because this comes directly from MongoDB's core engine, its `name` is usually `MongoServerError`, not `ValidationError`. However, it always attaches the numeric property `error.code = 11000`.

2. Why the `||` Check Is Necessary

If your `catch` block only looks for Mongoose validation:

```
// FLAWED:
if (error.name === 'ValidationError') {
  return res.status(400)...
}
return res.status(500)...
```

A duplicate email will fail the `if` check because `error.name` is `MongoServerError`, falling straight through to the `500 Internal Server Error` response.

To automated test suites, a duplicate email is considered invalid user input (a client fault), so tests expect a **`400 Bad Request`**, not a `500`. Checking `error.code === 11000` intercepts that specific database rejection and maps it to `400`.

3. Why It Is Preferred Over Pre-Checking

Checking inside the `catch` block avoids the "Check-Then-Act" race condition:

- If you do a pre-check with `User.findOne({ email: req.body.email })`, two simultaneous requests with the same email can both pass the pre-check at the exact same millisecond before either writes to the database.
- With Option A, you delegate the uniqueness guarantee to MongoDB's atomic index lock. If a collision occurs, MongoDB safely rejects the second write, and your `catch` block converts that rejection into a clean `400` response.

User prompt: Okay. Thank you for the help.

Response: You're very welcome! Good luck putting the final touches on your `PUT` routes and wrapping up the assignment. If any weird edge cases or test suite quirks pop up as you test the full pipeline, feel free to reach back out!
